



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Развој Cloud апликација у паметним мрежама

Приручник
- Чиста архитектура и DDD -
2025/2026

Садржај

Увод.....	2
Архитектурни обрасци и принципи	2
Чиста архитектура (Clean Architecture).....	2
Слојеви	2
Правила зависности.....	3
Инверзија контроле и инјекција зависности	4
Дизајн вођен доменом (Domain-Driven Design)	5
Основни концепти	5
Апликациони слој и проток података	6
CQRS (Command Query Responsibility Segregation).....	6
MediatR (медијатор образац)	6
Доменски догађаји наспрам апликационих догађаја	7
Комуникација у реалном времену.....	7
Чист код (Clean code)	8
Закључак	8

Увод

Овај документ даје детаљан преглед архитектуре и технологија које су коришћене на вежбама. Систем је дизајниран пратећи принципе чисте архитектуре (*Clean Architecture*) и дизајн вођен доменом (*Domain-Driven Design*), уз коришћење *cloud-native* сервиса на *Microsoft Azure* платформи.

Архитектурни обрасци и принципи

Чиста архитектура (Clean Architecture)

Чиста архитектура представља софтверски **архитектурни образац** који се ослања на строгој подели система на **слојеве** и **независности пословне логике** од **техничких детаља имплементације**.

Формализована је од стране Robert C. Martin (познатији као Uncle Bob), а њена кључна карактеристика је **правило зависности** (*Dependency Rule*):

Све зависности морају бити усмерене ка унутра, односно ка језгру апликације!

Циљ је креирати систем који је:

- Независан од оквира (*framework*),
- Лак за тестирање,
- Независан од корисничког интерфејса (*UI*),
- Независан од базе података,
- Независан од спољних интеграција (*API*),
- Јасно подељене одговорности делова система.

Слојеви

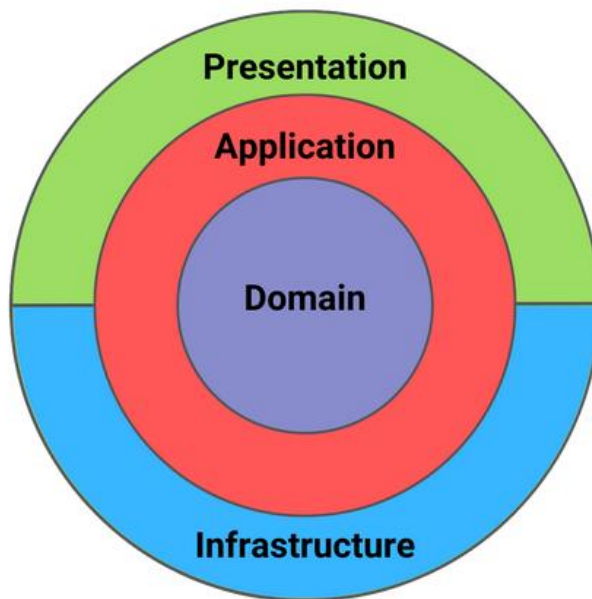
Чиста архитектура има следеће слојеве:

1. **Домен/језгро** (*Domain Layer*)
Најнижи слој који **не зависи** ни од једног другог слоја или технологије. Служи да дефинише пословну логику која је заједничка за читав систем.
 2. **Случајеви коришћења** (*Application Layer*)
Слој који зависи само од доменског слоја. Служи за **оркестрирање извршавања** пословне логике без бриге о томе како се подаци чувају или приказују.
 3. **Технички детаљи** (*Infrastructure Layer*)
-

Слој који зависи од апликационог слоја како би имплементирао његове интерфејсе. Садржи конкретне имплементације репозиторијума, сервиса, интеграција са спољним сервисима итд... Његова улога је решавање техничких питања, са "прљавим" кодом који зависи од **специфичне библиотеке**.

4. Улазна тачка (*Presentation Layer*)

Слој који прима захтеве, претвара их у формат који апликациони слој разуме и враћа кориснику одговор. Садржи контролере, аутентификационе и ауторизационе middleware и друге.



Слика 1 Визуелна репрезентација чисте архитектуре

Правила зависности

Приликом имплементације треба обратити пажњу на следећа правила зависности:

- Домен не сме да зна за базу података, *HTTP*, *Azure*, оквир који се користи итд..
Конкретно не сме да садржи NuGet пакете који уводе техничке зависности и референце на друге пројекта. Могуће је додати "чисте пакете" нпр. за сложене математичке прорачуне.,
- Апликациони слој не зна за базу података, *HTTP*, *Azure* и остале технологије које се користе . **Сме да садржи само референцу на домен и библиотеке за валидацију и оркестрацију** (нпр. *FluentValidation*, *MediatR*, *AutoMapper* итд...). Никако не сме да садржи библиотеке које уводе *Azure SDK*, *HTTP Client*, *EF Core* итд...

- Инфраструктура зависи од апликационог и доменског слоја и имплементира интерфејсе коришћењем **конкретних технологија** за комуникацију са базом (нпр. *EF Core*), рад са *Azure Storage*, комуникација са екстерним сервисима итд... **Промена имплементације у овом слоју не сме да наруши рад остатка система** (нпр. прелазак са *Azure* на *AWS*). У коликој мери је то заиста могуће зависи искључиво колико је квалитетна апстракција у апликационом слоју.

Пример: Уколико се приликом дефинисања интерфејса за репозиторијум у апликационом слоју открију детаљи имплементације како ради *EF Core (ORM)* нпр. враћа *IQueryable<T>*, прелазак на *Dapper (micro-ORM)* у том случају неће бити могућ без измене самог интерфејса!

- Презентациони слој зависи од апликационог и доменског слоја, али не сме да зависи од слоја инфраструктуре. Изузетак овом правилу је када у презентационом слоју треба да убацимо *DI* конфигурацију. Ово се ради тако што у слоју инфраструктуре дефинишемо екстензионе методе (нпр. *AddInfrastructure()*) које касније позивамо у презентационом слоју.

Инверзија контроле и инјекција зависности

Чиста архитектура подразумева примену принципа инверзије контроле (*Inversion Of Control - IoC*) и инјекцију зависности (*Dependency Injection DI*). Ови концепти омогућавају поштовање правила зависности и јасно раздвајање одговорности између слојева.

Инверзија контроле представља принцип по коме класе не креирају саме своје зависности, већ им се те зависности испоручују споља.

Инјекција зависности је конкретан механизам којим се *IoC* реализује. Уместо да класа сама креира зависности, она их прима (најчешће) преко конструктора.

IoC и **DI** су практични начини да се **DIP** (*Dependency Inversion Principle*) принцип (део **SOLID**) примени у коду!

DIP: Модули високог нивоа не смеју зависити од модула ниског нивоа. Оба треба да зависе од апстракције.

У контексту чисте инфраструктуре **интерфејси** се дефинишу у апликационом слоју (у изузетним случајевима могу и у домену), **имплементирају** у слоју инфраструктуре, а **DI контејнер** се дефинише у презентационом слоју. **Презентациони слој одлучује која конкретна имплементација ће бит повезана са одговарајућом апстракцијом и тиме заправо "саставља" систем.**

Зашто интерфејси нису у слоју инфраструктуре?

- Јер би тада апликациони слој зависио од инфраструктуре и нарушио поделу одговорност слојева,
- Бизнис логика би постала зависна од технологије.

Дизајн вођен доменом (Domain-Driven Design)

Дизајн вођен доменом је приступ развоју софтвера који ставља **пословни домен у центар система**. За разлику од традиционалних приступа развоју:

- Не почиње се од шеме базе података
- Не почиње се од избора технологија,
- Не почиње се од REST API интерфејса,
- Не почиње од CRUD операција,
- **Почиње се од разумевања доменског знања и проблема који се решава!**

Основни концепти

Заједнички језик (*Ubiquitous Language*)

Програмери и доменски стручњаци користе **исту терминологију** у коду, документацији и разговору.

Ентитети (*Entity*)

Ентитети су објекти који имају **идентитет**, односно могу се јединствено идентификовати у систему, и чији се идентитет не мења током целог животног циклуса, чак и ако се сви остали атрибути промене. Не служе само за чување података, већ диктирају шта се са тим подацима може одрадiti.

За разлику од традиционалног приступа где су ентитети само "врећа за смештање података", са јавним *get* и *set* својствима (тзв. анемични модели), у DDD они постају тзв. богати модели. То значи да ентитет сам не дозвољава да дође у невалидно стање. **Пословна логика више није разбацана по екстерним сервисима, већ се налази унутар самог ентитета.**

Вредносни објекти (*Value Object*)

Објекти који се дефинишу са вредношћу, немају идентитет. Ако су вредности једнаке, објекти се сматрају једнаким.

Агрегат (*Aggregate*) и коренски агрегат (*Aggregate Root*)

Агрегат је логичка група ентитета и вредносних објеката који се третирају као једна недељива целина ради очувања конзистентности података.

Коренски агрегат је главни ентитет кроз који се једино сме приступити осталима елементима унутар агрегата. Његов задатак је да заштити унутрашње стање система од невалидних промена и осигура да се ниједно пословно правило не прекрши.

Доменски догађаји (*Domain Events*)

Доменски догађај представља запис о нечему значајном што се већ догодило у пословном домену. Због тога се њихова имена увек пишу у прошлом времену (нпр. *OrderCreated*). Док се ентитети и вредносни објекти баве **тренутним стањем** система, доменски догађаји се баве **променом** тога стања.

Напомена: Основни концепти DDD-а имплементирани су у решењу пројекта. За ентитет је коришћен директоријум *models*, јер Microsoft у свом екосистему користи појам *entity* у контексту модела који се чувају у бази података. Да би се задржала доследност овој нотацији:

- У слоју инфраструктуре се користи термин **ентитет** (чува се у бази података),
- У домену се користи термин **доменски модел** (ентитет према дефиницији DDD).

Апликациони слој и проток података

Након дефинисања домена и његових граница, дефинише се апликациони слој који преузима улогу оркестратора. На вежбама смо користили CQRS шаблон и библиотеку *MediatR*.

CQRS (Command Query Responsibility Segregation)

CQRS је образац који раздваја операције **читања** од операција **уписа/измене** података:

- **Команде** (*Commands*): представљају намере корисника да промене стање система. Команде мењају стање и не враћају податке (осим можда идентификатора),
- **Упити** (*Queries*): служе искључиво за читање података. Упити никада не мењају стање система и оптимизовани су за брз и лак приступ подацима.

MediatR (медијатор образац)

Како бисмо имплементирали CQRS без директног везивања контролера са сервисом користимо библиотеку **MediatR**. Она омогућава да систем функционише путем порука.

- **Захтев** (*Request*): објекат који носи податке (команда или упит),

- **Обрађивач (Handler):** Изолована класа која садржи логику извршавања специфичног захтева.

У пројекту са вежбама је коришћен приступ писања команде или упита у једном фајлу (*vertical slice*), где постоје три дела:

- **Команда:** DTO објекат који прослеђује презентациони слој,
- **Валидатор:** Имплементиран помоћу библиотеке *FluentValidation*. Он осигурава да су улазни подаци исправни пре него што стигну до обрађивача,
- **Обрађивач:** Садржи чисту логику оркестрације – позива доменске моделе, врши измене, позива репозиторијуме.

MediatR Pipeline Behaviors је пресретац који је конфигурисан да пре позивања обрађивача провери податке из команде и спречи даље извршавање ако валидација не прође. Без њега би се у сваком обрађивачу ручно позивала валидација.

Доменски догађаји наспрам апликационих догађаја

У решењу са вежби користимо два типа догађаја:

- **Доменски догађаји (Domain Events)**
Настају унутар доменског слоја (унутар агрегата или ентитета) и представљају специфичну промену пословног стања (нпр. *OrderCreated*). Обрађују се помоћу *INotification* механизма.
- **Апликациони догађаји (Application Events)**
Настају у апликационом слоју и генерише га *MediatR* обрађивач. Служе за интеграцију са спољним системима или покретање позадинских процеса.

Комуникација у реалном времену

За разлику од класичног веб модела у коме клијент иницира сваки HTTP захтев, овај систем захтева могућност да сервер **самостално** и **тренутно** обавести клијенте о променама у систему. Промене стања клијенту морају да буду доступне без потребе да се уради освежавање странице или да се ради *polling*.

SignalR је ASP .NET библиотека која омогућава **двосмерну комуникацију** између сервера и клијента у **реалном времену**.

DeviceHub је централна комуникациона тачка (*Hub*) система, имплементирана помоћу *SignalR* библиотеке. Налази се у презентационом слоју и служи за обавештавање клијента о значајним догађајима слањем порука. Подржава следеће начине комуникације:

- **Broadcast:** слање поруке свим повезаним клијентима,
-

- **Групе:** слање поруке одређеној групи корисника,
- **Појединачни корисници:** слање поруке конкретном кориснику.

Чист код (Clean code)

Док нам чиста архитектура и дизајн вођен доменом пружају **макро-слику** и одређују како су компоненте система организоване, принципи **чистог кода** нам дају **микро-слику** – правила за писање кода унутар самих фајлова. Како би систем био лак за одржавање, архитектура мора бити подржана квалитетно и читљиво написаним кодом.

Приликом писања кода на вежбама, а и приликом израде самог пројекта, потребно је придржавати се следећих правила:

- **Именованье** (*Meaningful Names*) – класе, методе и променљиве морају имати **дескриптивна** имена која јасно и недвосмислено описују њихову намеру. **Забрањено** је коришћење нејасних скраћеница,
- **Методе** – треба да буду кратке и да раде само једну ствар, строго поштујући принцип јединствене одговорности (*Single Responsibility*). Треба избегавати велики број улазних параметара у потпису методе. Пракса је да се групишу и прослеђују кроз DTO,
- **Коментари** – код треба да буде самодокументујући. То значи да добро структурирани код са јасним именима не захтева додатна објашњења. Коментари не смеју да препричавају шта код ради, из самог кода треба да се разуме шта се дешава.
- **Обрада грешака** – За враћање резултата користити *Result Pattern*. **Забрањено је бацање изузетака!** Изузеци се не користе за контролу тока пословне логике, већ искључиво за неочекиване и критичне грешке,
- **DRY** (*Don't Repeat Yourself*) – сваки део пословне логике треба да постоји на једном месту у систему. Тиме се смањује ризик од грешака,
- **KISS** (*Keep It Simple, Stupid*) – увек треба тежити најједноставнијем решењу које испуњава захтеве. Избегавати прекомерно компликовање система.

Закључак

Примена принципа чисте архитектуре, дизајна вођеног доменом и правила чистог кода може се на почетку чинити комплексном и захтевати више времена за почетно подешавање система и организовање структуре фајлова. Међутим, овакав приступ дугорочно доноси предности у виду система који је отпоран на промене технологије, лак за тестирање и једноставан за одржавање након што пројекат порасте.